# News Popularity Prediction Using XGBoost: A Binary Classification Approach

2176387 Yunwoo Choi
2276305 Seojeong Cho
2276188 Ingyeong Yang
2491002 Jiwon Kang

# 1. Introduction

This project addresses a binary classification problem that aims to predict whether an online news article is popular (y=1) or not (y=0), based on its metadata. An article is considered popular (y=1) if it has been shared more than 1,400 times; otherwise, it is classified as not popular (y=0).

The dataset used is the Online News Popularity Dataset from the UCI Machine Learning Repository, which contains a wide range of article-related features. It is structured as follows

Training data (train.csv)

- 22,200 samples
- 49 total features
- Composition: 1 ID variable, 46 input variables, and 2 output variables (number of shares and binary label y)

Test data (test.csv)

- 9,515 samples
- 47 total features
- Number of shares and target variable (y) are not provided

The final objective is to train a model using the training data and generate prediction results (prediction.csv) for the test data. The evaluation metric is a combined score derived by averaging the following three metrics, allowing for a comprehensive assessment of the model's classification performance:

- Accuracy
- F1 Score
- AUC (Area Under the Curve)

# 2. Data Preprocessing

Data preprocessing was carried out to enhance model performance and ensure the reliability of prediction results. The main steps included handling missing values, encoding categorical variables, and selecting key features.

## 2.1 Key Feature Description

Eight key variables were selected and analyzed based on their potential influence on article popularity. Among them, some were actually used in model training.

- Category and Day of the Week: ‘`data_channel`’, ‘`weekday`’
    - `data_channel` (the article’s category) reflects the topic (ex. politics, sports, technology), which can significantly impact reader interest and popularity.
      `weekday` (the day of publication) captures differences in user traffic between weekdays and weekends, providing meaningful information for predicting article views.
- Sentiment Analysis: ‘`title_sentiment_polarity`’, ‘`abs_title_sentiment_polarity`’
    - `title_sentiment_polarity` represents the sentiment score of the article title, helping to assess how a positive or negative tone may influence click-through rates.
    - ‘`abs_title_sentiment_polarity`’ measures sentiment intensity, reflecting the possibility that emotionally charged titles, whether positive or negative,may attract more clicks compared to neutral ones.
- Visual Content: ‘`num_imgs`’, ‘`num_videos`’
    - `num_imgs` (number of images) and `num_videos` (number of videos) can draw readers’ attention visually and are especially influential in content sharing on social media, making them important indicators of popularity.
- Text Length: ‘`n_tokens_content`’, ‘`n_tokens_title`’
    - `n_tokens_content` (content length) reflects the depth and amount of information in the article; extremely short or long articles can affect reader retention.

    - `n_tokens_title` (title length) can be used to balance information delivery and click-through appeal, closely tied to popularity.

These features encompass topic, sentiment, visual elements, and content length, making them critical clues in predicting article popularity.

## 2.2 Handling Missing Values

- Sentiment Variables
    - `'title_sentiment_polarity'`, `'abs_title_sentiment_polarity'`
    - Missing values likely result from unavailable or failed sentiment analysis.
    - Replaced with `0.0` to represent neutral sentiment, as using the mean could distort analysis consistency.

- Image and Video Variables
    - `'num_imgs'`, `'num_videos'`
    - Missing values interpreted as absence of content → replaced with `0`
    - Using the mean could misrepresent content characteristics.

- Content and Title Length
    - `'n_tokens_content'`, `'n_tokens_title'`
    - These values can vary widely; median imputation minimizes the impact of outliers while preserving informational content.
    - Either median or mean was used depending on the variable's distribution.

- Day of the Week
    - `'weekday'`
    - Missing values replaced with the mode (most frequent value)
    - Since `weekday` is a categorical variable, mode imputation is appropriate and reflects general user behavior patterns.

- Article Category
    - `'data_channel'`

- If no clear information was available, replaced with `Other` to minimize information loss during encoding.

### 2.3 Encoding Categorical Variables

- Variables: ‘`weekday`’, ‘`data_channel`’, ‘`is_weekend`’
  These variables can reflect temporal and topical patterns in reader demand.

- Method: One-Hot Encoding
  - Converts each category into a binary vector.
  - Avoids assigning artificial ordinal relationships between categories, leading to more interpretable models.
  - Effective for representing categorical variables when the number of categories is not excessively large.
  - Encoding methods can be adjusted later depending on the model's characteristics.

## 3. RandomForestClassifier

### 3.1 Data Preprocessing

#### 3.1.1 Handling Missing Values for Numerical Features

- The following features were imputed using the median:
  `'n_tokens_title'`, `'n_tokens_content'`, `'n_unique_tokens'`, `'n_non_stop_words'`, `'n_non_stop_unique_tokens'`, `'num_hrefs'`, `'num_self_hrefs'`, `'num_imgs'`, `'num_videos'`
- Sentiment-related variables were filled with 0.0 when missing:
  `'title_sentiment_polarity'`, `'abs_title_sentiment_polarity'`
- Features requiring log transformation were first imputed with the mean, then transformed using `log1p`:
  `'self_reference_avg_sharess'`, `'num_keywords'`

### 3.1.2 Keyword-related Features

- Features starting with `'kw_'` were filled with their mean values.
- These features are considered critical as they reflect content themes and trending topics.

### 3.1.3 Categorical Variable Processing

- `data_channel` missing values were replaced with 'Other'.
- `weekday` missing values were filled with the mode (most frequent value).
- A new binary feature `is_weekend` was created:
  If the weekday is 'Saturday' or 'Sunday', set to 1, otherwise 0.
- Both categorical variables were One-Hot Encoded.

### 3.1.4 Dropping Irrelevant Features

- Columns not useful for training were removed:
  ‘`id`’, ‘`url`’, ‘`timedelta`’, ‘`shares`’.
- Any remaining missing values were dropped after preprocessing.

## 3.2 Data Splitting and Scaling

### 3.2.1 Train-Validation Split

- The dataset was split into 80% training and 20% validation sets.
- `random_state=42` was set to ensure reproducibility.

### 3.2.2 Standardization

- StandardScaler was applied to scale features to have zero mean and unit variance.
- The scaler was fit on the training set and transformed on the validation set only.

## 3.3 Model Design and Selection

### 3.3.1 Chosen Model: RandomForestClassifier

- An ensemble learning method using multiple decision trees to aggregate predictions.

- Known for its robustness to overfitting and effectiveness with both numerical and categorical data.
- Provides practical benefits such as variance reduction and feature importance interpretability.

### 3.3.2 Hyperparameter Tuning

- GridSearchCV was used with 3-fold cross-validation to optimize the F1 Score.

```
param_grid = {
    'n_estimators': [180, 190, 200, 203],
    'max_depth': [23, 22, 21, None],
    'min_samples_split': [9, 10, 11],
    'max_features': ['sqrt', 'log2']
}
```

- Hyperparameter search space:
    - `n_estimators`: 180, 190, 200, 203
    - `max_depth`: 23, 22, 21, None
    - `min_samples_split`: 9, 10, 11
    - `max_features`: `'sqrt'`, `'log2'`

### 3.3.3 Threshold Adjustment

- By default, Scikit-learn classifiers use a threshold of 0.5 for binary classification.
- However, to better capture the positive class (likely shared articles) and improve F1 Score, the threshold was adjusted.
- Applied thresholds: `0.48, 0.5, 0.52`
- Slightly lowering the threshold (ex. 0.48) helped to capture more relevant articles with marginal improvement in F1 Score.

### 3.3.4 Evaluation Metrics

- Evaluated using Accuracy, F1 Score, and AUC.
- The final model performance was measured as the average of these three metrics (Final Average Score).

```
Fitting 3 folds for each of 48 candidates, totalling 144 fits

📌 Best Parameters from GridSearchCV:
{'max_depth': 22, 'max_features': 'log2', 'min_samples_split': 10, 'n_estimators': 200}

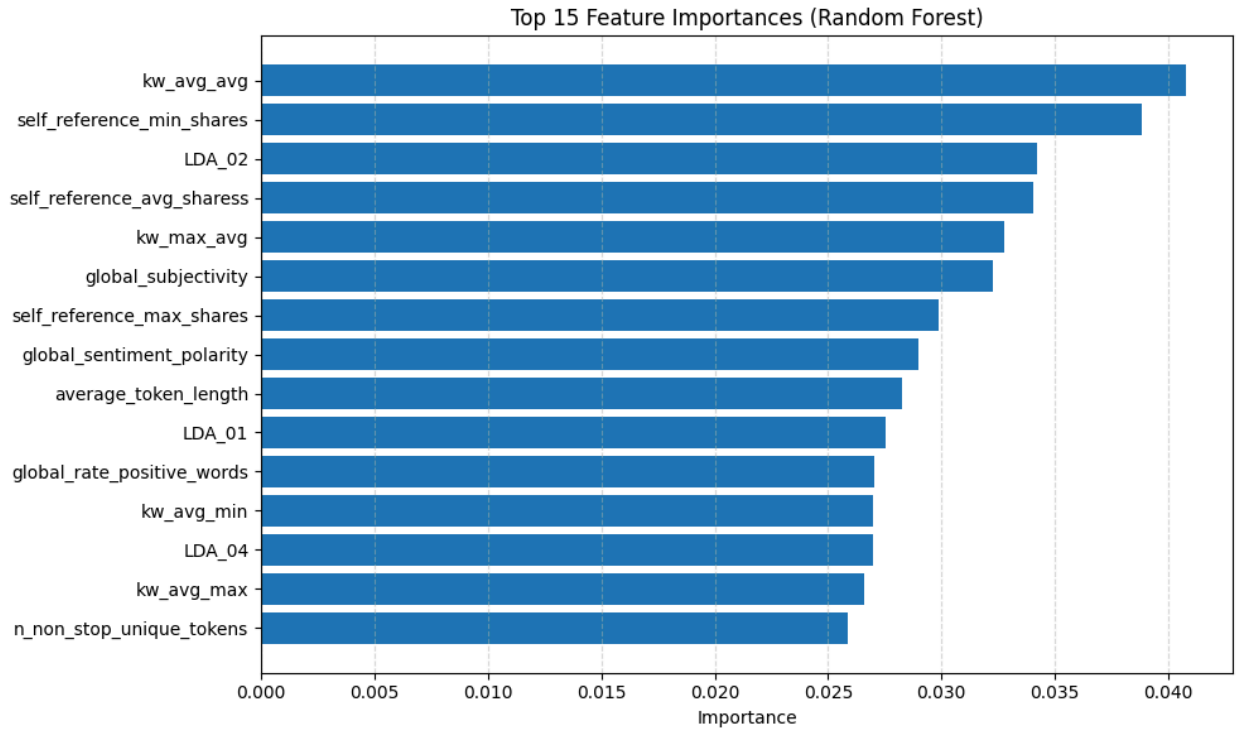
🔎 Threshold = 0.48
Accuracy: 0.6788, F1 Score: 0.6846, AUC: 0.7520
Final Score (avg): 0.7051

🔎 Threshold = 0.5
Accuracy: 0.6834, F1 Score: 0.6729, AUC: 0.7520
Final Score (avg): 0.7028

🔎 Threshold = 0.52
Accuracy: 0.6834, F1 Score: 0.6463, AUC: 0.7520
Final Score (avg): 0.6939
```

## 3.4 Feature Importance Analysis

- Feature importance was derived from the trained Random Forest model.
- High importance was assigned to keyword features, LDA topic distributions, and sentiment indicators.
- This suggests that the semantic, emotional, and topical content of an article is more influential than its length or structure in predicting shareability.



| feature | description |
| --- | --- |
| kw_avg_avg | Average keyword score in the article → Reflects the potential of capturing content quality and current trends. |
| LDA_02, LDA_01, LDA_04, LDA_00 | LDA topic distribution → Indicates that topic-related patterns play a significant role in predictive performance. |
| global_subjectivity | Subjectivity score of the full article → Suggests that more subjective articles are more likely to be shared. |
| kw_max_avg, kw_avg_min, kw_avg_max | Minimum/maximum/average keyword scores → Highlights the importance of diversity and quality in keyword usage. |
| self_reference_avg_sharess, self_reference_min_shares | Average and minimum self-reference shares → Demonstrates the usefulness of historical sharing data. |
| n_unique_tokens, n_non_stop_unique_tokens | Lexical diversity → Greater word variety may lead to increased reader engagement and interest. |
| global_rate_positive_words, global_sentiment_polarity | Ratio of positive words and overall sentiment score → Suggests a potential correlation between emotional tone and content sharing, supporting emotional marketing strategies. |

Interpretation:
The results confirm that semantic value and emotional relevance play a more significant role in user sharing behavior than superficial characteristics like article length or format.

## 3.5 Performance Limitations and Future Directions

Summary:

- RandomForestClassifier delivered the best performance among tested models, making it suitable for interim submission.
- Achieved a Final Average Score of ~0.705 on the training set but only ~0.66 on the test set.

### 3.5.1 Analysis of Limitations

- Model Capacity Limitation:
  While Random Forest is powerful, it may struggle to model complex, high-dimensional, non-linear interactions, especially with features like LDA and sentiment scores.
- Lack of Feature Engineering:
  Only pre-existing features were used. No interaction terms (ex., `kw_avg_avg x global_subjectivity`) or domain-informed features (ex. clustering-based) were added.
- Imbalanced Classes:
  The target variable may be imbalanced, leading to suboptimal performance even after threshold tuning.
- Threshold Adjustment Constraints:
  Although lowering the threshold to 0.48 slightly improved F1 Score, this strategy showed limited overall effect.
  More systematic approaches, such as using ROC curve-based threshold optimization, could be explored.

# 4. XGBClassifier

## 4.1 Data preprocessing process

### 4.1.1 Numeric variable preprocessing

- Outlier handling
    - Only numeric columns are searched
    - Find values that are out of the 1~99% range and cut them off with lower/upper limits
- Missing value handling
    - Replace missing values of sentiment features with 0.0
    - Replace missing values of features starting with ‘num’ with 0.0
    - Replace missing values of remaining features with median

### 4.1.2 Categorical variable preprocessing

- Missing values of ‘`data_channel`’ are replaced with ‘Others’
- Missing values of ‘`weekday`’ are replaced with mode
    - Derive weekend status (is_weekend) as 0/1
- One-hot encoding categorical variables with pd.get_dummies

### 4.1.3 Removing unnecessary columns

- Since the number of observations in the train data is greater than the number of input features, we think that the possibility of overfitting is low, so we use all features except ‘`id`’, ‘`shares`’, and ‘`n_non_stop_words`’ for training
- Remove unnecessary for model training column
    - ‘`id`’, ‘`shares`’, ‘`n_non_stop_words`’

## 4.2 Data Splitting

- Shuffle the data randomly and use 25% of the entire dataset as test data.

- Fix ‘`random_state`’ to ensure reproducible results.

## 4.3 Model Design and Selection

### 4.3.1 Selected Model: XGBClassifier

- Gradient tree-boosting algorithm with built-in overfitting prevention techniques
- Tree-based ensemble approach
- Overfitting can be controlled via regularization and parameters such as ‘`max_depth`’ and ‘`min_child_weight`’.

### 4.3.2 Hyperparameter Optimization

- Use 5-fold cross-validation
- Evaluation metric: F1 score

```
model = XGBClassifier(
  objective='binary:logistic',
  gamma = 0.3,
  max_depth = 5,
  n_jobs=-1,
  random_state=random_state,
)

param_grid = {
  'min_child_weight':  [1, 7],
  'learning_rate':     [0.05, 0.2],
  'n_estimators':      [25, 50, 100],
  'reg_alpha':         [0, 0.1, 1],
  'reg_lambda':        [0.1, 1, 100],
}
```

    - Hyperparameter Search Parameters

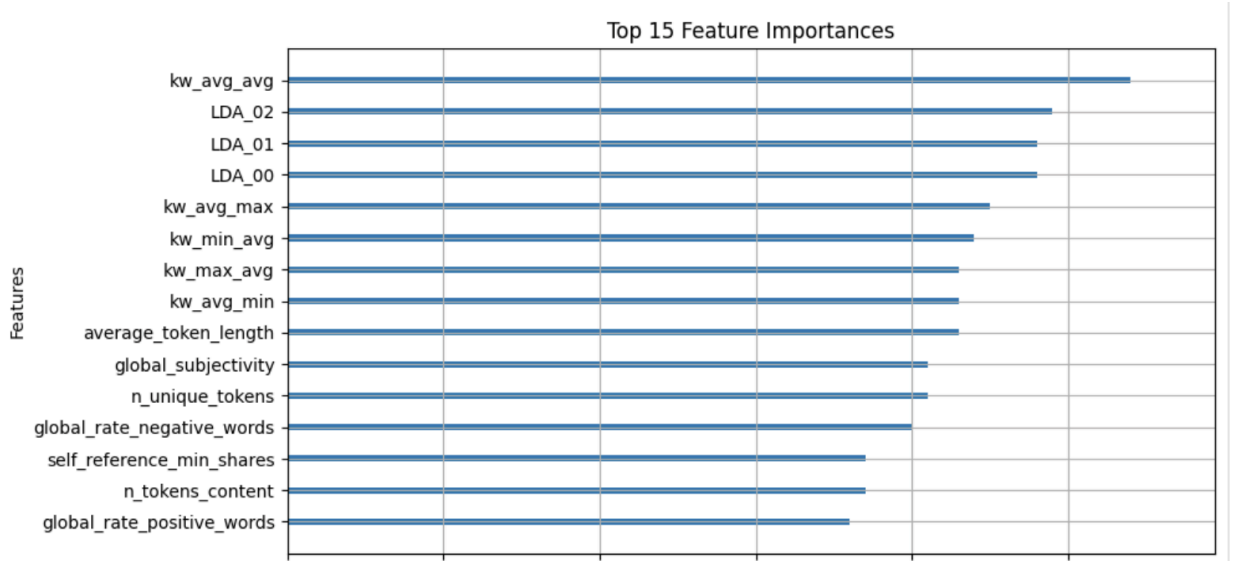
- `min_child_weight` (0–∞): Minimum sum of instance weights (or minimum number of observations) required in a leaf; smaller values can lead to overfitting.
- `learning_rate` (0–1): Step size shrinkage used in update to prevent overfitting; larger values may increase the risk of overfitting.
- `n_estimators`: Number of trees (boosting rounds).
- `reg_alpha`: L1 regularization term on weights; larger values reduce overfitting.
- `reg_lambda`: L2 regularization term on weights; larger values reduce overfitting.

### 4.3.3 Threshold

- The default classification threshold is usually 0.5 → adjust the threshold from 0.5 to find the optimal value → 0.414.

## 4.4 Feature Importance Analysis

We examined how much each feature contributed to the model's predictions.
Below are the top 15 features ranked by importance.




| Feature | Description |
|---|---|

| | |
|---|---|
| kw_avg_avg, kw_avg_max, kw_min_avg, kw_max_avg, kw_avg_min | Among meta keywords, the number of keyword sets has a large impact on an article's share |
| LDA_02, LDA_01, LDA_00 | The similarity between the topic and the article is correlated with share count. |
| averge_token_length | Length of the average word affects the shares of articles |
| global_subjectivity | The subjectivity of an article affects how much it is shared. |
| n_unique_tokens, n_tokens_content | The number of words and the proportion of unique words affect the shareability of an article |
| global_rate_negative_words | The proportion of negative words in an article affects how much the article is shared. |
| self_reference_min_shares | The number of citations to a less-shared article affects how much the article is shared. |
| global_rate_positive_words | The percentage of positive words in an article affects how much the article is shared. |

## 4.5 Performance Limitations

### 4.5.1 Model Expressiveness Limitations

- Lack of complexity control
    - Since sampling/feature complexity control hyperparameters were not used, it is difficult to regulate overfitting or underfitting

### 4.5.2 Feature Engineering Limitations

- All numerical variables were clipped to the 1–99% range, which may have removed important outliers
- Missing values were filled simply with 0.0 or the median, without using log transformation, square-root transformation, etc., to mitigate distribution skew
- No interaction features (ex. `weekday × data_channel` or `polarity × subjectivity`) were considered, making it difficult for the model to learn relationships between features

### 4.5.3 Insufficient Consideration of Class Imbalance

- Imbalance-handling metrics were not used, so the model's learning focus on the minority class was inadequate
- Only the F1-score was used in GridSearchCV, meaning no imbalance-focused metric was optimized

# 5. Discussion

This project addressed a binary classification task that predicts whether a given news article will be popular on social media based on its metadata. To this end, we conducted a series of experiments focusing on two tree-based models: RandomForestClassifier and XGBoostClassifier. We analyzed the models' performance, feature responsiveness, and the impact of preprocessing strategies from multiple perspectives.

## 5.1 Model Behavior and Predictive Characteristics

- RandomForestClassifier is an ensemble model that aggregates multiple decision trees using a bagging strategy. It is robust to overfitting and tends to perform well on datasets where feature interactions are relatively simple.
    - The experimental results showed that the Final Average Score reached approximately 0.705, demonstrating stable performance compared to the baseline model.
    - In particular, the model was highly responsive to the LDA-based topic distributions (ex. `LDA_02, LDA_01`), keyword scores (`kw_avg_avg,`

`kw_min_avg`), and global sentiment indicators such as `global_subjectivity`, effectively capturing structural and semantic patterns in the content.

- On the other hand, XGBoostClassifier uses gradient boosting, where each tree learns to correct the errors of its predecessor, thus gradually improving performance.
    - By adjusting the decision threshold from the default 0.5 to 0.414, we were able to improve Recall, achieving more balanced performance under an imbalanced class distribution.
    - Ultimately, XGBoost achieved an F1 Score of 0.7001, AUC of 0.7243, and Accuracy of 0.6587, resulting in a Final Average Score of 0.6944.

## 5.2 Feature Design Effects

The two models demonstrated differing sensitivities to features:

- RandomForest focused on keyword scores and LDA topic distributions.

- XGBoost showed strong responsiveness to sentiment features such as `title_sentiment_polarity and global_sentiment_polarity`.

This indicates that XGBoost has a better ability to capture subtle emotional patterns and nonlinear interactions.

## 5.3 Performance Limitations and Future Improvements

- RandomForestClassifier

    - Achieved a high Final Average Score of approximately 0.705 on the train set, which led to its selection as the interim submission model.

    - However, performance declined to 0.6619 on the test set, revealing a limitation in generalization.

- XGBoostClassifier

    - Scored a slightly lower 0.6944 on the train set, but showed consistent and stable performance of 0.6825 on the test set.

- As a result, XGBoost was determined to have superior generalization capabilities due to its smaller performance gap between train and test sets, and was selected as the final submission model.

This finding demonstrates the risk of evaluating models based solely on training performance, as it may lead to overestimated results that do not generalize well.

**Summary of Future Improvement Directions**

| Improvement Direction | Description |
|---|---|
| Interaction feature engineering | Ex. Multiply features like `kw_avg_avg` × `global_subjectivity` |
| Clustering-based feature design | Assign cluster numbers based on article type via unsupervised learning |
| Automated Threshold Optimization | Use ROC curve to automatically find optimal threshold |
| Model Ensemble Extension | Improve performance with ensemble techniques such as Voting and Stacking |
| Explainable AI | Adopt SHAP, LIME to enhance model interpretability |

Additionally, while only original features were used in this project, advanced feature engineering could improve future performance. Examples include:

- Creating ratios (ex. title-to-content word count)
- Categorizing sentiment scores
- Adding variables based on deviations from weekday-wise average share count

In the long term, transitioning to deep neural networks (DNNs) with two or more hidden layers may be worth exploring. This would enhance the model's capacity for nonlinear representation learning, allowing it to capture complex feature interactions and potentially improve predictive performance.

# 6. Conclusion

This project aimed to build a prediction model for news article popularity using the UCI Online News Popularity Dataset. Through various preprocessing and model experiments, we searched for an optimal modeling strategy.

Although both models achieved satisfactory performance, XGBoostClassifier demonstrated more robust and generalizable results.

- Final Model Chosen: XGBoostClassifier
- Final Performance Metrics:
    - Accuracy: 0.6587
    - F1 Score: 0.7001
    - AUC: 0.7243
    - Final Avg: 0.6944

In conclusion, feature design and threshold tuning proved to be more critical than model structure itself. XGBoost was particularly effective in adapting to such detailed tuning. Moving forward, we expect to enhance both predictive accuracy and interpretability by integrating explainable AI tools (ex. SHAP) and more sophisticated interaction and clustering-based features.